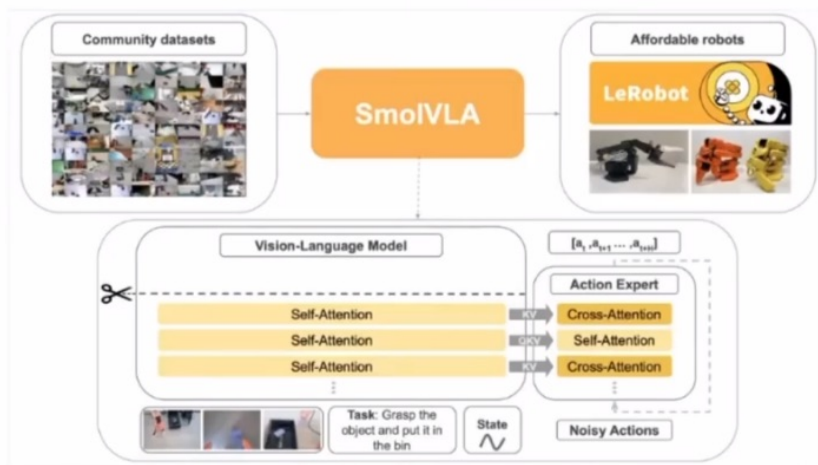


二、架构分析:

SmolVLA 先对数据进行预训练, 再根据特定任务进行后训练。模型架构方面, TF和流匹配进行结合¹ 舍弃了一半视觉模型, 采用交叉注意力机制和自注意力机制, 对速度和低延迟推理进行了优化; 引入异步推理堆栈, 将机器人执行动作的方式与它们理解所见所闻的方式分开, 可以在快速变化的环境中更快地做出反应。



2.1 主要架构:



2.1.1 视觉语言模型

SmolVLM2 (主干) --处理图像和语言的组合任务

- **SigLIP 视觉编码器**: “看”图像, 把图像内容转化为一种计算机能理解的表示形式, 即“图像令牌”。这些令牌就像图像的“数字化摘要”。
- **SmolLM2 语言解码器**: 处理语言指令 (比如“拿起杯子”), 并把它们转化成“语言令牌”。
- **额外输入**: 模型还会接收机器人的“感觉运动状态” (比如机器人手臂的位置、速度等)。这些状态通过一个简单的“线性层”被压缩成一个“状态令牌”, 确保它和语言、图像令牌的格式一致。
- **结果**: 图像、语言、状态令牌被串联起来, 送到SmolLM2解码器中(整合令牌), 解码器会处理它们, 生成一组特征 (“综合信息”)。这些特征随后被传递给“动作专家”来决定机器人该做什么。

本体感觉

2.1.2 动作专家

变压器 + 流量匹配策略

桌球TF

- 动作专家是一个紧凑的**变压器**(规模比SmolLM2 (主干中的语言和多模态解码器) 小, 专为动作生成任务优化, 一般是自注意力), 会根据VLM的输出, 生成机器人的动作序列。
- 流匹配: 一种生成模型的**训练方法**, 目标是让模型学会把“有噪声的动作”修正回“正确的动作”。这种方法会形成一个“平滑的向量场”, 帮助模型更稳定、准确。

总结流程

1. 输入:

- 图像通过SigLIP编码器变成**图像令牌**。
- 语言指令通过SmolLM2变成**语言令牌**。
- 机器人状态通过线性层变成**状态令牌**。

2. 处理:

- 这些令牌被串联后, 送到SmolLM2解码器, **生成综合特征**。

3. 动作生成:

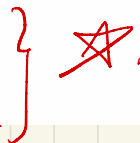
- 特征被送到动作专家 (小型变压器)。
- 动作专家用流匹配方法, 直接**预测连续的动作序列** (比如机器人的运动轨迹)

2.2 效率优化措施

很多VLA都是结合**VLM和动作专家模块**, 让机器人根据图像和指令执行动作, 但SmolVLA通过三个技术优化了效率, 整体上实现了**速度快、计算量小、动作稳健**

应用了三个关键技术:

- 减少视觉标记的数量**
- 跳过 VLM 中的上层**
- 以及在动作专家中交错交叉和自我注意力。**



2.2.1 视觉标记减少

高分辨率图像的问题:

图像被输入到模型前, 会被“切”成小块 (像拼图一样), 每块变成一个“标记” (token), 供模型处理。高分辨率图像 (比如512×512像素) 会产生很多标记 (比如1024个), 这会让模型处理得很慢, 因为计算量大。

引入了**视觉令牌缩减技术**, 通过**像素洗牌** (pixel shuffle) 将每帧视觉令牌限制为64个 (而非传统的1024个), 显著降低了计算负担。

SmolVLA处理方法

- 把标记数量限制到每张图像**64个**, 而不是1024个。
- 用了一种叫**PixelShuffle**的技术, 把大图像“压缩”成更少的标记 (就像把高分辨率照片压缩成低分辨率, 但保留关键信息)。
- 虽然模型在训练时用更详细的图像 (更多标记), 但在实际运行时, 只用“全局图像” (即一个简化的、低分辨率的图像特征集合), 这样速度更快。

2.2.2 跳过 VLM 中的上层

通过跳过层实现更快推理，VLM的层 视觉语言模型 (VLM) 是一个 **多层神经网络**，每层处理信息的方式不同。最后 **一层通常包含最“高级”的信息**，但计算最后一层需要经过前面所有层，耗时长。而且，有研究表明，中间层的信息有时候对具体任务（比如动作预测）更有用。

深层 $\rightarrow$ 抽象 浅层/中层更懂“物理”

SmolVLA处理方法：

- 不总是用VLM的最后一层，而是从**中间层提取信息**（比如总共20层，只用前10层）。
- 动作专家（Action Expert）直接用这些中间层的特征来生成动作，跳过了后面一半的层。

2.2.3 交错交叉和自我注意力

帮助模型决定“关注”哪些信息。比如，机器人要做动作时，需要关注图像、指令。

SmolVLA处理方法：

在**动作专家内使用**，动作专家内部的注意力层在两种模式间交替

- 交叉注意力（CA）：动作标记（action tokens）会“看”VLM的输出（图像+指令的信息），确保动作跟指令和环境匹配。比如，“拿起杯子”时，动作要根据杯子的位置来调整。
- 自我注意力（SA）：动作标记“看”**之前的动作标记**（因果关系，防止未来信息泄露），确保动作序列平滑。比如，机器人手臂移动时，不能突然跳跃，需要平滑过渡。
- 这些CA和SA层是**交错排列的**（比如CA-SA-CA-SA），而不是全用一种。

2.3 异步推理的创新

传统同步推理中，机器人需要等待模型完成**感知**（perception）、动作预测（action prediction）和动作执行（action execution）的整个流程。这种方式在快速变化的环境中会导致延迟，因为每一步都按顺序执行。

异步推理通过**解耦感知和动作预测与动作执行**，允许模型在机器人执行**前一组动作时**，同时处理新的观测数据并生成下一组动作（称为 **action chunk**）。这减少了推理调用的频率，提高了约 30% 的响应速度和 2 倍的任务吞吐量。

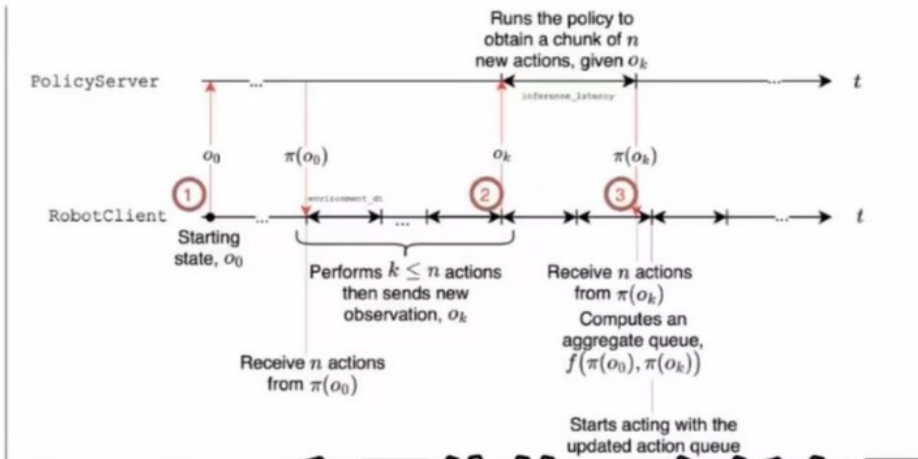


图 3.异步推理。异步推理堆栈的图示。请注意，该策略可以在远程服务器上运行，可能使用 GPU。

机器人要根据环境（比如图像、指令）生成动作块，推理（计算）这些动作块的方式有两种：

• **同步推理 (Sync) :**

- 机器人**先执行一个动作块**，完成后停下来，等待模型计算下一个动作块。
- **缺点：**反应慢，因为每次都要等新动作块计算出来，机器人才能继续动。

▼ • **异步推理 (Async) :** | I

- 机器人**一边执行当前动作块**，**一边**把最新的环境信息（比如新拍的图像）发给“策略服务器”（可以是远程 GPU）去计算下一个动作块。
- **优点：**没有空闲时间，机器人能更快响应环境变化。

SmolVLA的异步推理堆栈

SmolVLA使用异步推理，通过**三个关键机制**让机器人动作更流畅、更具适应性：

(1) 早期触发

- **机制：**当机器人执行动作块时，系统会监控“**动作队列**”（待执行的动作清单）的长度。如果队列快用完了（比如只剩70%的动作），系统会立刻把最新的环境观察（比如新图像、传感器数据）发送到策略服务器，触发下一个动作块的计算。

(2) 解耦线程

- **机制：**机器人的“动作执行”（实际移动手臂）和“推理”（计算下一个动作块）是分开的，跑在不同的线程上。推理过程不会阻塞动作执行，机器人可以一边动一边算。

▼ (3) 数据块融合 I

- **机制：**连续的动作块可能会有“重叠”（比如前一个块的最后动作和后一个块的第一个动作很接近）。SmolVLA用“合并规则”把这些重叠动作拼接起来，确保动作过渡平滑。

异步推理的优势

- **零执行滞后：**机器人不用停下来等新动作块，动作执行和推理同时进行，就像流水线作业。
- **更高适应性：**通过实时发送最新观察（图像、状态），机器人能快速应对环境变化（比如物体移动了）。
- **无需改模型：**这些优化是系统层面的，不需要改变模型本身，就能提升性能。
- **支持远程计算：**推理可以在强大的远程服务器（比如GPU）上跑，机器人只需要执行动作，适合资源有限的设备。